

ANÁLISIS EXPLORATORIO DE DATOS: Medidas de Resumen

UNIDAD 3 - Parte 1

Fundamentos de Ciencia de Datos 2026



VARIABLE

Una **variable** es una característica, cualidad o propiedad observada que puede asumir diferentes valores y es susceptible de ser cuantificada o medida en una investigación.

Las observaciones registradas de una o más variables conforman un **conjunto o set de datos**.

En el **formato largo**, cada fila del dataset constituye un registro y cada **columna** contiene la información correspondiente a una **variable**.

VARIABLES CUANTITATIVAS Y CUALITATIVAS

Cualitativas o categóricas: son aquellas variables que no son medibles numéricamente.

Cuantitativas: son aquellas que toman valores numéricos para los cuales tiene sentido pensar en operaciones aritméticas. Pueden ser:

- **Continuas:** pueden asumir teóricamente cualquier valor dentro de un intervalo.
- **Discretas:** sólo pueden tomar valores aislados en dicho intervalo.



Ejemplo

Para trabajar algunos conceptos de esta clase, utilizaremos datos de la **4ta. Encuesta Nacional de Factores de Riesgo 2018**, una encuesta llevada a cabo en 2018 por el INDEC y la Secretaría de Salud en la que se relevó información sobre condiciones de salud, hábitos y factores de riesgo en la población adulta argentina.

```
import pandas as pd

# Leemos datos e inspeccionamos las primeras filas del DataFrame
data = pd.read_csv('datasets/enfr2018.txt', delimiter = '|')

data.head()
```

	id	cod_provincia	region	tamano_aglomerado	aglomerado	localidad
0	1128639	2	1	1	1	1
1	1709939	2	1	1	1	1
2	6874130	2	1	1	1	1
3	10319375	2	1	1	1	1
4	11140857	2	1	1	1	1

5 rows × 7 columns

VARIABLES CUANTITATIVAS Y CUALITATIVAS

🧐 ¿Cómo clasificaríamos a cada una de las siguientes variables incluidas en el dataset?

- Provincia de origen
- Nivel de colesterol
- Número de ambientes/habitaciones de la vivienda
- Índice de masa corporal
- Edad en años cumplidos
- Nivel de instrucción (*sin instrucción, primario completo, ...*)
- Rango etario (*18 a 24 años, 25 a 34 años, ...*)

MEDIDAS O ESTADÍSTICAS DE POSICIÓN

MEDIDAS DE POSICIÓN

Las medidas de posición brindan información acerca de la localización del conjunto de observaciones de una variable

Las más comunes son:

- Media aritmética
- Fractilas o cuantilos
- Modo

MEDIA ARITMÉTICA ($\bar{X}$)

Es la suma de los valores observados de la variable X dividida por el número total de datos de dicha variable.

$$\bar{X} = \frac{1}{n} \sum_{i=1}^n X_i$$

Para calcularla, contamos con el método **mean()** de **Pandas**, que ya empleamos anteriormente.

MEDIA ARITMÉTICA

Tomemos como ejemplo la variable *ingreso total mensual del hogar* (**bhih01**):

```
media = data['bhih01'].mean().round(1)
print(f'Media aritmética: {media}')
```

Media aritmética: 22446.7

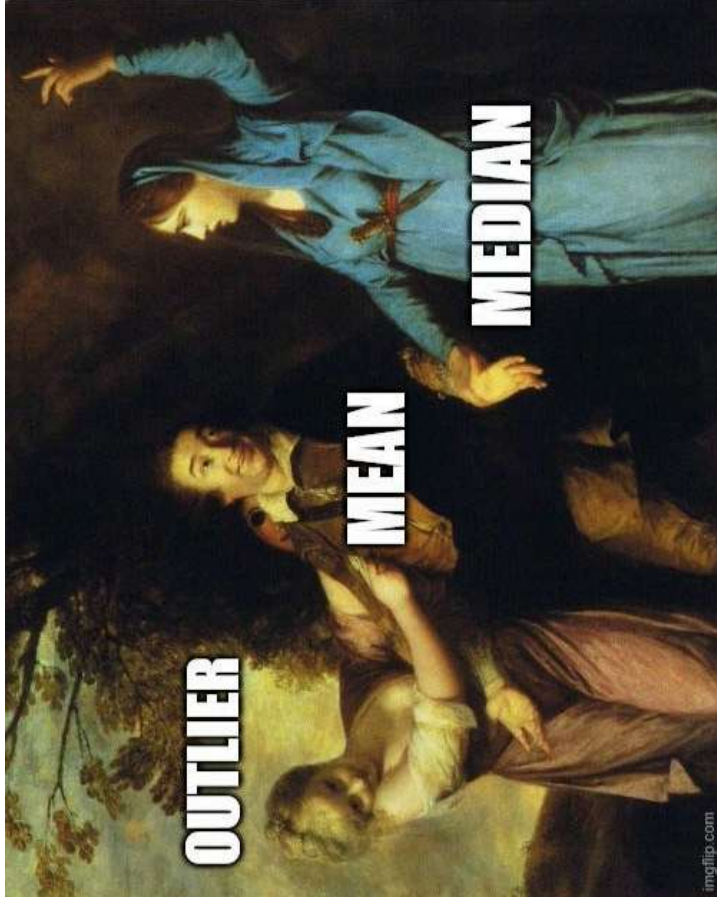
🧐 ¿Cómo interpretamos el valor obtenido en el contexto de los datos con los que estamos trabajando?

MEDIANA (Q₂)

La mediana, segundo cuartil o percentil del 50% se define como el valor de la variable que se encuentra en la **posición central** del conjunto de datos **ordenado de menor a mayor**. La mitad de las observaciones son menores o iguales que él y la otra mitad son mayores.

¿Cómo se calcula?

MEDIA VS. MEDIANA



Como la media **depende de todos los valores registrados de la variable**, la presencia de alguna observación anormalmente grande o pequeña (*outlier*) **influye sensiblemente en ella**.

En estas situaciones nos conviene buscar una medida de tendencia central que resulte **más representativa** del conjunto. La **mediana** es una buena alternativa.

MEDIA TRUNCADA (TRIMMED MEAN)

De manera general, es un promedio calculado sobre los datos una vez que **se ha eliminado un cierto porcentaje de las observaciones más pequeñas y más grandes del conjunto** ($\alpha \cdot 100\%$). Es una medida intermedia entre la media y la mediana.

MEDIA TRUNCADA (TRIMMED MEAN)

De manera general, es un promedio calculado sobre los datos una vez que **se ha eliminado un cierto porcentaje de las observaciones más pequeñas y más grandes del conjunto** ($\alpha \cdot 100\%$). Es una medida intermedia entre la media y la mediana.



Es el método oficial utilizado en la evaluación de pruebas olímpicas de patinaje sobre hielo y varias otras actividades deportivas y artísticas.

FRACTILAS O CUANTILOS

La mediana es un tipo particular de medida que pertenece a una clase mucho más general de estadísticas descriptivas de centralidad: **las fractilas o cuantilos**.

En términos generales, la **fractila de orden r** es aquel valor de la variable tal que el **$r\%$** de las observaciones del **conjunto ordenado de datos** son menores o iguales a él.

CUARTILOS (Q_1 , Q_2 , Q_3)

Los cuartilos son un tipo de fractila que divide al conjunto de datos ordenados de la variable en **4 partes con aproximadamente el mismo número de datos**:

- **Cuartil 1 (Q_1)**: es el valor tal que el **25%** de los datos son menores o iguales a él.
- **Cuartil 2 (Q_2)**: es el valor tal que el **50%** de los datos son menores o iguales a él. **Coincide con la mediana**, definida anteriormente.
- **Cuartil 3 (Q_3)**: es el valor tal que el **75%** de los datos son menores o iguales a él.

CÁLCULO DE LOS CUARTILOS

Pandas cuenta con el método **quantile()**, que permite calcular cualquier fractila que se desee. La misma puede especificarse en el parámetro **q**, que por *default* es igual a 0.5.

Otro parámetro importante es **interpolation**, que permite definir el **método a utilizar para estimar la fractila**. Las distintas posibilidades pueden consultarse en la [documentación de la función](#) y difieren en la forma en que se calcula la posición y en cómo se interpola entre valores adyacentes cuando la posición no es un entero.

Por *default*, **Pandas** utiliza **interpolation = 'linear'**, también conocido como **Método R-7**.

MÉTODO R-7

Según este método, la posición Pos de la fractila Q ($0 \leq Q \leq 1$) se calcula de la siguiente manera:

$$Pos = 1 + Q(n - 1)$$

Luego, la fractila se calcula teniendo en cuenta la parte fraccional de $Pos(f)$ y las observaciones que, en el conjunto ordenado, se encuentran en las posiciones enteras alrededor de $Pos(x_i$ y x_j , con $x_i \leq x_j$):

$$x_Q = x_i + f(x_j - x_i)$$

MÉTODO R-7 (versión “a mano”)

Por ejemplo, si contamos con los siguientes datos para la variable X : [2, 2, 4, 5, 5, 6, 7, 8] y queremos calcular el primer cuartil (Q_1), primero obtenemos la posición:

$$Pos = 1 + 0.25(8 - 1) = 2.75$$

Luego, como es un valor que se encuentra entre las observaciones ubicadas en las posiciones 2 y 3 del conjunto, el valor de Q_1 resulta:

$$Q_1 = 2 + 0.75(4 - 2) = 3.5$$

MÉTODO R-7 (versión pandas)

```
# Definimos una Serie con los valores de X
serie = pd.Series([2, 2, 4, 5, 5, 6, 7, 8])

# Calculamos el primer cuartil
q1 = serie.quantile(q = 0.25, interpolation = 'linear')

# Imprimimos el resultado
print('Primer cuartil:', q1)
```

Primer cuartil: 3.5

OTROS MÉTODOS

Otras opciones para **interpolation** son:

- **interpolation** = **'lower'**: toma x_i
- **interpolation** = **'higher'**: toma x_j
- **interpolation** = **'nearest'**: toma el más cercano entre x_i y x_j
- **interpolation** = **'midpoint'**: toma el punto medio entre x_i y x_j

CÁLCULO DE LOS CUARTILOS

Calculemos los cuartilos del conjunto de observaciones de la variable *ingreso total mensual del hogar* (**bhih01**):

```
# Calculamos los tres cuartilos
cuartilos = data['bhih01'].quantile(q = [0.25, 0.5, 0.75])

# Imprimimos la Serie
print(cuartilos)
```

```
0.25    10000.0
0.50    18000.0
0.75    30000.0
```

```
Name: bhih01, dtype: float64
```

 ¿Cómo podríamos interpretar estos valores?

DECILOS Y PERCENTILOS

Otras fractilas que pueden definirse:

- **Decilos:** son valores de la variable que dividen al conjunto de observaciones en **diez partes** con aproximadamente el mismo número de datos.
- **Percentilos:** dividen al conjunto de observaciones de la variable en **cien partes** con aproximadamente el mismo número de datos.

CÁLCULO DE FRACTILAS (versión **numpy**)

numpy cuenta con diversas funciones para calcular fractilas. Un ejemplo es **percentile()**, en la que la fractila a calcular se debe indicar como un número entre 0 y 100 (el correspondiente %) en el parámetro **q**.

```
import numpy as np

# Calculamos la mediana del ingreso total mensual por hogar
np.percentile(data['bhih01'], q = 50)

18000.0
```

También permite seleccionar el método de estimación a utilizar en el parámetro **method** (por *default* se utiliza una interpolación lineal).

MODA

La moda es el valor de la variable que se presenta **mayor número de veces**, es decir, el que tiene la **mayor frecuencia**.

Podemos calcular la moda de las observaciones del *ingreso total mensual por hogar* utilizando el método **mode()** de Pandas:

```
data['bhih01'].mode()  
0    20000  
Name: bhih01, dtype: int64
```

🤖 ¿Cuál es el resultado? ¿Cómo podríamos interpretar el valor obtenido?

MODA

Tenemos que tener presente que podemos tener conjuntos de observaciones que presenten **más de una moda**. En estos casos, la Serie de **pandas** que siempre devuelve el método **mode()** como *output* estará formada por tantos elementos como modas haya en el conjunto:

```
serie_ejemplo = pd.Series([2, 3.5, 8, 3.5, 4, 6.5, 8, 6.5])

serie_ejemplo.mode()

0    3.5
1    6.5
2    8.0
dtype: float64
```

MODA

La moda es la única medida de posición que puede usarse para datos provenientes de una **variable cualitativa**.

Si tomamos como ejemplo la variable ***tipo de vivienda*** (**bhcv01**), las respuestas posibles según la encuesta fueron: *casa, casilla, departamento, pieza de inquilinato, pieza en hotel o pensión*, entre otras.

MODA

Podemos calcular la moda de las observaciones de esta variable para analizar cuál fue el tipo de vivienda más frecuente entre los hogares encuestados:

```
data['bhcv01'].mode()  
0      1  
Name: bhcv01, dtype: int64
```

🤖 ¿Qué significa este valor?

MODA

Si el resultado es 1, esto no significa que “1” sea el tipo de vivienda más frecuente en sí mismo, sino que **la categoría codificada como 1 es la más frecuente**. En este caso, según el diccionario de variables:

1 = casa, 2 = casilla, 3 = departamento, ...

Por lo tanto, la categoría más frecuente entre los hogares encuestados es *casa*.

MEDIDAS O ESTADÍSTICAS DE DISPERSIÓN

MEDIDAS DE DISPERSIÓN

Las medidas de posición son útiles pero resumen **sólo parte** de la información contenida en el conjunto de datos. Dos conjuntos de observaciones pueden tener aprox. la misma media y mediana pero diferir en cuánto se “alejan” del valor central.

Surge la necesidad de definir **medidas de dispersión que describan la variabilidad de los datos.**

En esta Unidad trabajaremos con: **desvío estándar/variancia, rango, rango intercuartil, coeficiente de variación y desviación mediana absoluta.**

RANGO

El rango es la diferencia entre el mayor y el menor valor observado de la variable.

$$Rango = X_{max} - X_{min}$$

Para el *ingreso total mensual por hogar*, es igual a:

```
# Calculamos el rango
rango = data['bhih01'].max() - data['bhih01'].min()

# Imprimimos el resultado
print(f'Rango: {rango}')
```

Rango: 420000

VARIANCIA (S^2)

Es una medida de cuánto se desvían, en promedio, las observaciones de una variable respecto a la media aritmética.

$$S^2 = \frac{1}{n-1} \sum_{i=1}^n (X_i - \bar{X})^2$$

A mayor variabilidad, mayor S^2 .

DESVIACIÓN ESTÁNDAR (S)

Se define como la raíz cuadrada positiva de la variancia.

$$S = + \sqrt{\frac{1}{n-1} \sum_{i=1}^n (X_i - \bar{X})^2}$$

En comparación con la variancia, posee la ventaja de que está expresada en las unidades de la variable, por lo que su interpretación es más sencilla y directa.

Podemos interpretarla como una “**distancia promedio de las observaciones con respecto a la media**”.

CÁLCULO DE VAR. Y DESVÍO ESTÁNDAR

Pandas cuenta con los métodos **var()** y **std()** que permiten calcular variancia y desvío estándar, respectivamente.

```
# Calculamos el desvío estándar del ingreso total mensual por hogar
desv_est = round(data['bhih01'].std(),1)

# Calculamos la variancia del ingreso total mensual por hogar
variancia = round(data['bhih01'].var(),1)

# Imprimimos los resultados
print(f'Desvío estándar: {desv_est}')
print(f'Variancia: {variancia}')
```

Desvío estándar: 19756.6

Variancia: 390322722.2

COEFICIENTE DE VARIACIÓN (CV)

Es el desvío estándar dividido por la media aritmética.

$$CV = \frac{S}{|\bar{X}|}$$

Es una medida adimensional que indica **qué proporción representa el desvío estándar de la media aritmética.**

Se suele utilizar cuando se desea comparar la variabilidad de dos o más conjuntos de datos que difieren en unidades de medida y/o magnitudes.

RANGO INTERCUARTIL (RI)

Es la diferencia entre el tercer y el primer cuartil y, como tal, **mide la dispersión del 50% de los datos centrales.**

$$RI = Q_3 - Q_1$$

Es una medida de dispersión que **no está influenciada por valores extremos**. Por este motivo, cuando se usa la mediana como medida de centralidad, el RI es la medida de dispersión adecuada para acompañarla.

RANGO INTERCUARTIL (RI)

Calculamos el RI para las observaciones de la variable *ingreso total mensual por hogar*:

```
# Calculamos el rango intercuartil para el ingreso total mensual por hogar
iqr = cuartilos[0.75] - cuartilos[0.25]

# Imprimimos el resultado
print(f'Rango intercuartil: {iqr}')
```

Rango intercuartil: 20000.0

 ¿Cómo podemos interpretar el valor obtenido?

DESVIACIÓN MEDIANA ABSOLUTA (MAD)

Es una versión robusta del desvío estándar basada en la mediana, que se define como la **mediana de los valores absolutos de las diferencias entre cada observación y la mediana**.

DESVIACIÓN MEDIANA ABSOLUTA (MAD)



DESVIACIÓN MEDIANA ABSOLUTA (MAD)

Es una versión robusta del desvío estándar basada en la mediana, que se define como la **mediana de los valores absolutos de las diferencias entre cada observación y la mediana**.

$$MAD = Mediana(|X_i - Q_2|)$$

DESVIACIÓN MEDIANA ABSOLUTA (MAD)

Podemos obtenerla de la siguiente manera:

```
# Calculamos las diferencias absolutas entre cada observación y la mediana
data['ingreso_dif_abs'] = abs(data['bhih01'] - cuartilos[0.5])

# La MAD es la mediana de las observaciones de 'peso_dif_abs'
mad = round(data['ingreso_dif_abs'].quantile(0.5), 2)

# Imprimimos el resultado
print(f'Desviación mediana absoluta: {mad}')
```

Desviación mediana absoluta: 9000.0

EL MÉTODO `describe()`

Una forma fácil y rápida de obtener gran parte de las métricas vistas hasta acá es utilizar el método **`describe()`** sobre la columna del **DataFrame** de interés.

```
data['bhih01'].describe()

count    29224.000000
mean      22446.653846
std       19756.586805
min         0.000000
25%      10000.000000
50%      18000.000000
75%      30000.000000
max      42000.000000
Name: bhih01, dtype: float64
```

EL MÉTODO `describe()`

¿Qué información obtenemos al aplicarlo sobre una columna que contiene datos de una **variable cualitativa**? Probemos con la variable *tipo de vivienda* (**bhcv01**):

```
data['bhcv01'].describe()
```

```
count    29224.000000
mean      1.307692
std       0.751898
min       1.000000
25%       1.000000
50%       1.000000
75%       1.000000
max       7.000000
Name: bhcv01, dtype: float64
```

 ¿Tiene sentido esta salida?

EL MÉTODO `describe()`

Aunque conceptualmente se trata de una variable cualitativa, su tipo de dato en el DataFrame no es categórico:

```
data['bhcv01'].dtype  
dtype('int64')
```

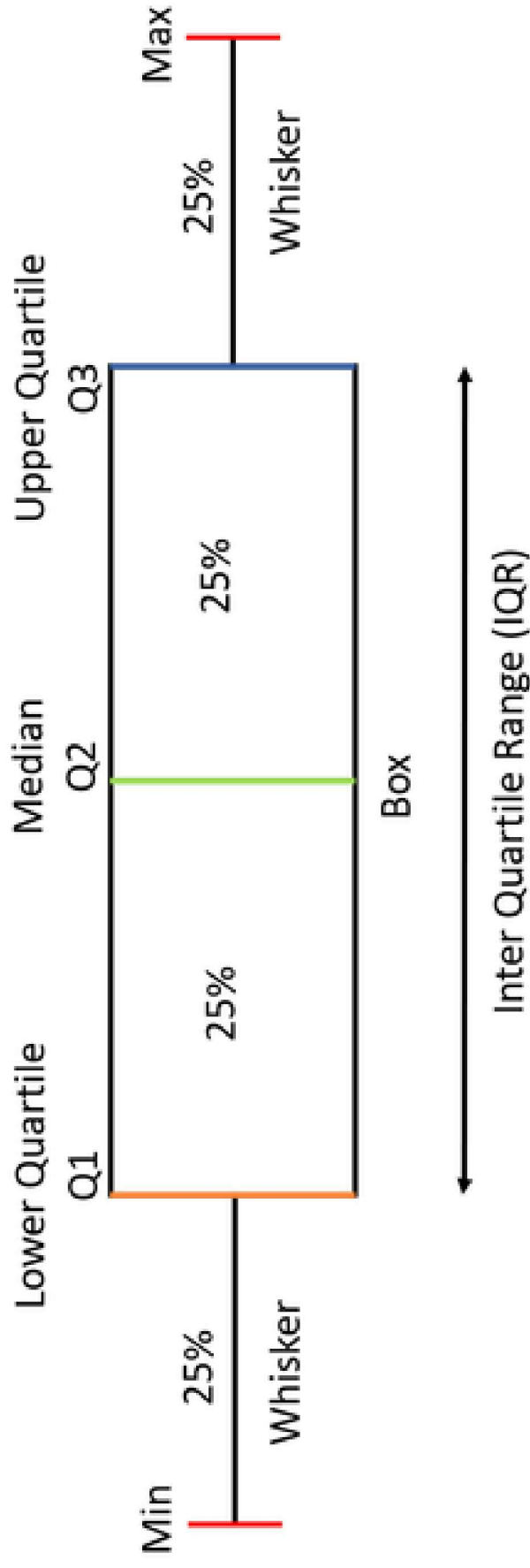
Luego de recodificarla y almacenar el resultado en una nueva columna (**tipo_vivienda**), la salida de `describe()` es la siguiente:

```
data['tipo_vivienda'].describe()  
  
count      29224  
unique         7  
top          Casa  
freq       24746  
Name: tipo_vivienda, dtype: object
```

EL BOXPLOT

EL BOXPLOT

Los cuartiles y los valores mínimo y máximo conforman un conjunto de cinco números que brindan un buen resumen de nuestros datos y con los cuales podemos construir un gráfico llamado **boxplot**.



EL BOXPLOT MODIFICADO

Una versión modificada del boxplot permite detectar **potenciales outliers**, es decir, **observaciones que no son típicas del conjunto**.

- Se consideran **potenciales outliers** aquellas observaciones que caigan por fuera de $(Q_1 - 1.5RI)$ y $(Q_3 + 1.5RI)$.
- La modificación del gráfico consiste en **extender los whiskers** hasta las observaciones mínima y máxima **que no sean puntos atípicos**. Los **outliers** se marcan en el gráfico como **puntos** separados de los *whiskers*.

EL BOXPLOT MODIFICADO



Ejemplo

Para ilustrar el boxplot modificado, trabajaremos con la variable ***minutos semanales de actividad física intensa*** (según el diccionario de registros de la ENFR 2018, se trata de actividades que hacen respirar mucho más rápido, exigen mayor esfuerzo físico y aceleran el ritmo cardíaco).

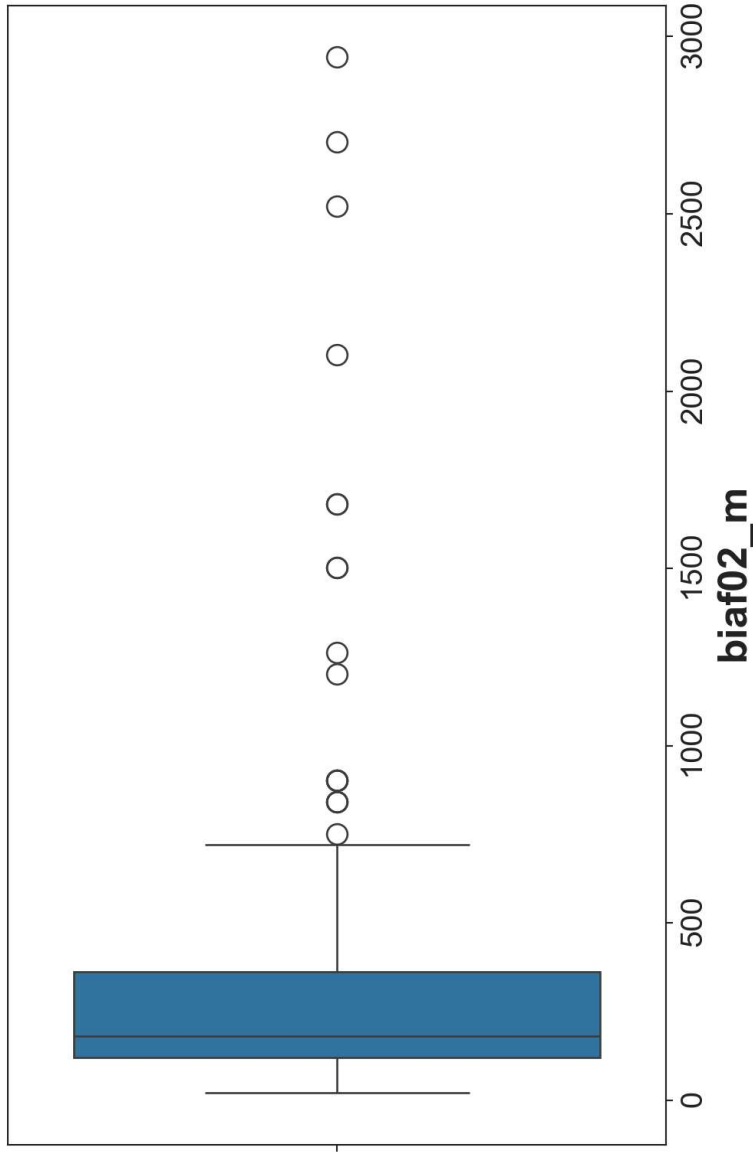
Filtraremos únicamente las observaciones correspondientes a la provincia de Santa Fe.

```
data_stafe = data[data['cod_provincia'] == 82]
```

EL BOXPLOT MODIFICADO

```
import matplotlib.pyplot as plt
import seaborn as sns

sns.boxplot(x = 'biaf02_m', data = data_stafe)
```



En base al gráfico obtenido:

🤔 ¿Hay potenciales *outliers* entre las observaciones de la variable *minutos semanales de actividad física intensa*?

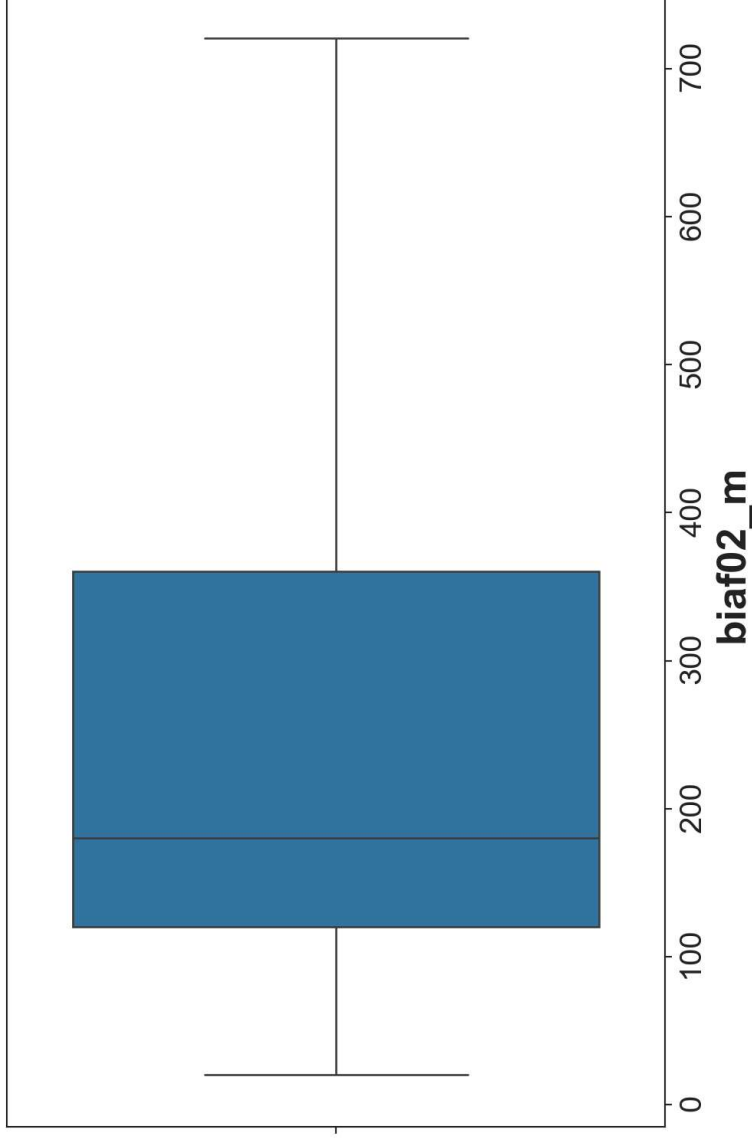
🤔 ¿Qué puede decirse acerca de la **simetría** del conjunto de observaciones de dicha variable?

En base a las respuestas a las preguntas anteriores, ¿qué medidas de posición y de dispersión serían las más adecuadas para describir a estos datos?

BOXPLOT SIN PLOTEO DE OUTLIERS

Si seteamos el parámetro **showflilers = False**, los potenciales *outliers* ya no se muestran en el gráfico resultante (**ipero es como si esos datos no existieran!**):

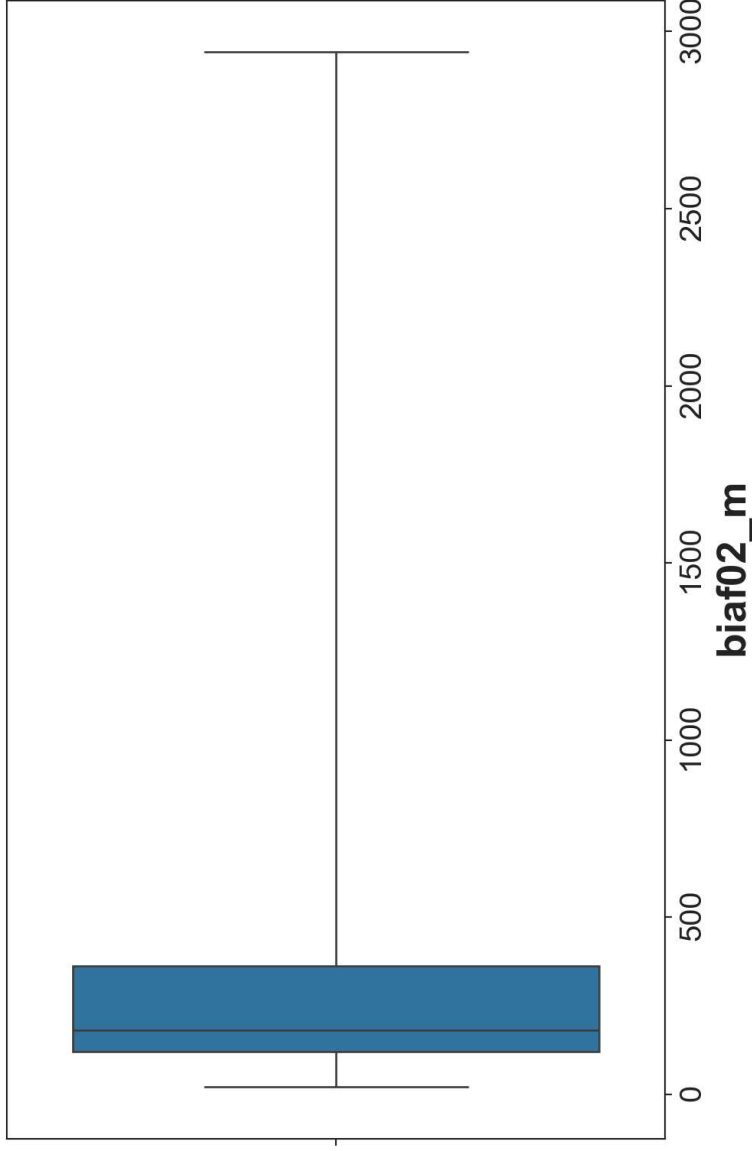
```
sns.boxplot(x = 'biaf02_m', showflilers = False, data = data_stafe)
```



BOXPLOT SIN PLOTEO DE *OUTLIERS*

Una forma de construir el **boxplot clásico** (el que no muestra potenciales *outliers*) sin borrar datos, es modificar el parámetro ***whis***:

```
sns.boxplot(x = 'biaf02_m', whis = [0,100], data = data_stafe)
```



¿QUÉ SON REALMENTE LOS *OUTLIERS*?

Un *outlier* es, estrictamente, una observación atípica según un criterio estadístico.

El criterio que definimos anteriormente:

Considerar *outlier* toda observación que caiga por fuera del intervalo $(Q_1 - 1.5RI, Q_3 + 1.5RI)$

fue popularizado por el estadístico estadounidense John Tukey en el marco del análisis exploratorio de datos, y su propósito es **identificar observaciones inusuales en el contexto de un conjunto de datos de una variable.**

EL ERROR MÁS COMÚN



Importante

Eliminar observaciones únicamente por el hecho de ser *outliers* puede alterar la distribución original de los datos y sesgar los resultados del análisis.

En muchos contextos, los valores extremos son completamente válidos y forman parte del fenómeno bajo estudio.

¿QUÉ HACEMOS CON LOS *OUTLIERS*?

Ante la detección de un valor atípico, lo adecuado no es eliminarlo automáticamente, sino preguntarse:

- ¿Se trata de un error de medición o de carga?
- ¿Es un valor posible dentro del fenómeno estudiado?
- ¿El objetivo del análisis justifica conservarlo o tratarlo de manera especial?

¿QUÉ HACEMOS CON LOS OUTLIERS?

¿Cuándo puede justificarse excluir un outlier?

Si bien los valores atípicos no deben eliminarse automáticamente, **existen situaciones en las que su exclusión puede estar justificada**. En particular, cuando hay evidencia de que el valor:

- corresponde a un error de carga o digitación,
- es físicamente imposible,
- proviene de una falla de medición documentada.

En estos casos, la exclusión no se basa en que el valor sea “extremo”, sino en que **existe una razón sustantiva o técnica para considerarlo inválido**.

¿QUÉ HACEMOS CON LOS *OUTLIERS*?

¿Y si no hay motivo para considerarlo inválido?

Puede ocurrir que, aun siendo correcto, un valor extremo tenga una influencia desproporcionada sobre los valores de ciertas estadísticas, como la media aritmética o las estimaciones de los coeficientes de un modelo estadístico (Unidad 6).

En esos casos, en lugar de eliminarlo sin más, es recomendable:

- analizar el resultado con y sin esa observación,
- utilizar medidas más robustas (como la mediana)
- aplicar métodos estadísticos diseñados para reducir la influencia de valores extremos.

TABLA DE FRECUENCIAS

TABLA DE FRECUENCIAS

Una tabla de frecuencias constituye una forma sencilla y efectiva para resumir la información de las observaciones de una variable de un dataset.

En el caso de que la variable en cuestión sea **cualitativa**, la tabla contiene las diferentes categorías de la misma junto con el número de veces que se presentó cada una de ellas (**frecuencia absoluta**) en el dataset.

También suele utilizarse la **frecuencia relativa**, que se define como el cociente entre la correspondiente frecuencia absoluta y el número total de datos.

TABLA DE FRECUENCIAS

Puede construirse fácilmente utilizando el método **value_counts()** de **Pandas**.

La versión más simple de una tabla de frecuencias para la variable *tipo de vivienda* (**tipo_vivienda**) es la siguiente:

```
data['tipo_vivienda'].value_counts()
```

tipo_vivienda	
Casa	24746
Departamento	3992
Casilla	312
Pieza de inquilinato	79
Pieza en hotel o pensión	49
Otros	33
Local no construido para habitación	13
Name: count, dtype: int64	

TABLA DE FRECUENCIAS

Podemos emprolijarla un poco y darle más forma:

```
tabla_viviendas = data['tipo_vivienda'].value_counts().reset_index().rename  
print(tabla_viviendas)
```

Tipo de vivienda	Frec_absoluta
Casa	24746
Departamento	3992
Casilla	312
Pieza de inquilinato	79
Pieza en hotel o pensión	49
Otros	33
Local no construido para habitación	13

TABLA DE FRECUENCIAS

A la tabla anterior, podemos agregarle una columna que contenga la **frecuencia relativa** de cada una de las actividades físicas:

```
# La frecuencia relativa siempre es un número entre 0 y 1
tabla_viviendas['Frec_relativa'] = round(tabla_viviendas['Frec_absoluta']/
print(tabla_viviendas)
```

Tipo de vivienda	Frec_absoluta	Frec_relativa
Casa	24746	0.85
Departamento	3992	0.14
Casilla	312	0.01
Pieza de inquilinato	79	0.00
Pieza en hotel o pensión	49	0.00
Otros	33	0.00
Local no construido para habitación	13	0.00

TABLA DE FRECUENCIAS

Otra alternativa hubiera sido expresar las frecuencias anteriores como **porcentajes sobre el total de registros**:

```
tabla_viviendas['Porcentaje'] = round(tabla_viviendas['Frec_absoluta'] * 100 /  
print(tabla_viviendas)
```

Tipo de vivienda	Frec_absoluta	Porcentaje
Casa	24746	84.7
Departamento	3992	13.7
Casilla	312	1.1
Pieza de inquilinato	79	0.3
Pieza en hotel o pensión	49	0.2
Otros	33	0.1
Local no construido para habitación	13	0.0

TABLA DE FRECUENCIAS

En el caso de variables **cuantitativas discretas**, aun cuando el número de observaciones sea grande, si la **cantidad de valores diferentes es relativamente baja** es posible construir una tabla de características similares para resumir la información.

Por ejemplo, en nuestro dataset de la ENFR 2018 tenemos la variable *cantidad de miembros del hogar encuestado* (**cant_componentes**).

```
data['cant_componentes'].unique()  
array([ 2,  3,  1,  4,  6,  5,  7,  8, 10,  9, 11, 14, 12, 13, 15],  
      dtype=int64)
```

TABLA DE FRECUENCIAS

```
# Tabla de frecuencias para la variable 'cantidad de miembros' ordenada por
tabla_cant_miembros = data['cant_componentes'].value_counts().sort_index()

print(tabla_cant_miembros)
```

	Frec_absoluta
Cant. de miembros	
1	6630
2	7760
3	5664
4	4786
5	2458
6	1102
7	459
c	nan

TABLA DE FRECUENCIAS

Cuando trabajamos con variables que **pueden asumir un gran número de valores diferentes**, lo que ocurre generalmente en el caso de variables **cuantitativas continuas**, en lugar de construir la tabla considerando los valores individuales que las mismas pueden asumir, se hace un **agrupamiento previo en subintervalos (segmentación)**.

TABLA DE FRECUENCIAS

Por ejemplo, para el *total de ingresos mensuales por hogar* (**bhih01**) en la Provincia de Santa Fe, cuyas estadísticas descriptivas son:

```
data_stafe['bhih01'].describe()

count      1773.000000
mean       24070.896785
std        22206.270573
min         0.000000
25%        10000.000000
50%        20000.000000
75%        30000.000000
max        350000.000000
Name: bhih01, dtype: float64
```

TABLA DE FRECUENCIAS

...podríamos realizar un agrupamiento de los valores observados en intervalos de igual amplitud, por ejemplo de 35000 pesos:

- De 0 a 35000 pesos: **[0, 35000)**
- De 35000 a 70000 pesos: **[35000, 70000)**
- De 70000 a 105000 pesos: **[70000, 105000)**
- ...
- De 280000 a 315000 pesos: **[280000, 315000)**
- De 315000 a 350000 pesos: **[315000, 350000)**
- De 350000 a 385000 pesos: **[350000, 385000)**

TABLA DE FRECUENCIAS

Para eso, podemos utilizar el método **pd.cut()**:

```
# Bins o puntos de corte
bins_ingresos = np.arange(0, 390000, 35000)

# Generamos los subintervalos con pd.cut()
data['ingreso_seg'] = pd.cut(data['bhih01'], bins = bins_ingresos, right =
data['ingreso_seg'].head(4)
```

```
0    [35000, 70000)
1    [35000, 70000)
2    [35000, 70000)
3    [70000, 105000)
```

Name: ingreso_seg, dtype: category

```
Categories (11, interval[int64, left]): [[0, 35000) < [35000, 70000) <
[70000, 105000) < [105000, 140000) ... [245000, 280000) < [280000,
315000) < [315000, 350000) < [350000, 385000)]
```

TABLA DE FRECUENCIAS

```
# Utilizamos value_counts() para generar la tabla de frecuencias
tabla_ingresos = data['ingreso_seg'].value_counts().sort_index()

print(tabla_ingresos)
```

```
ingreso_seg
[0, 35000)      23802
[35000, 70000)   4553
[70000, 105000)  727
[105000, 140000)   61
[140000, 175000)   42
[175000, 210000)   17
[210000, 245000)    2
[245000, 280000)    6
[280000, 315000)    1
```

TABLA DE FRECUENCIAS

```
# Formateamos nuestra tabla
tabla_ingresos = tabla_ingresos.reset_index().rename(columns = {'ingreso_s
print(tabla_ingresos)
```

Ingreso total mensual	Frec_absoluta
[0, 35000)	23802
[35000, 70000)	4553
[70000, 105000)	727
[105000, 140000)	61
[140000, 175000)	42
[175000, 210000)	17
[210000, 245000)	2
[245000, 280000)	0

TABLA DE FRECUENCIAS

Podemos agregar una columna que contenga la **frecuencia relativa** correspondiente a cada subintervalo de ingreso total mensual:

```
tabla_ingresos['Frec_relativa'] = round(tabla_ingresos['Frec_absoluta']/su  
print(tabla_ingresos)
```

Ingreso total mensual	Frec_absoluta	Frec_relativa
[0, 35000)	23802	0.81
[35000, 70000)	4553	0.16
[70000, 105000)	727	0.02
[105000, 140000)	61	0.00
[140000, 175000)	42	0.00
[175000, 210000)	17	0.00
[210000, 245000)	2	0.00
~~~~~	~	~

# TABLA DE FRECUENCIAS

A la tabla anterior podemos agregarle las **frecuencias absoluta y relativa acumuladas** correspondiente a cada subintervalo de distancias, utilizando el método **cumsum()**.

La **frecuencia absoluta acumulada** es el número de observaciones de la variable menores al límite superior del subintervalo correspondiente.

La **frecuencia relativa acumulada** es el cociente entre la correspondiente frecuencia absoluta acumulada y el número total de datos.

# TABLA DE FRECUENCIAS

```
# Agregamos ambas frecuencias acumuladas
tabla_ingresos['Frec_absoluta_cum'] = tabla_ingresos['Frec_absoluta'].cumsum()
tabla_ingresos['Frec_relativa_cum'] = tabla_ingresos['Frec_relativa'].cumsum()
```

La fila de la tabla correspondiente al intervalo [35000, 70000) es la siguiente:

```
tabla_ingresos.iloc[1]
Frec_absoluta      4553.00
Frec_relativa      0.16
Frec_absoluta_cum  28355.00
Frec_relativa_cum   0.97
Name: [35000, 70000), dtype: float64
```

# TABLA DE FRECUENCIAS



tabla_ingresos.iloc[1]	
Frec_absoluta	4553.00
Frec_relativa	0.16
Frec_absoluta_cum	28355.00
Frec_relativa_cum	0.97
Name: [35000, 70000), dtype: float64	

🧐 Para el intervalo [35000, 70000), ¿cómo se interpreta cada una de las cuatro frecuencias?